Matrix Based Password Storage

Traditional Methods

Traditionally, passwords are stored in databases as hashed values. During authentication, the system uses the username to locate the corresponding record and compare the stored hash with the hash of the entered password. (OWASP, 2025)

Hashes

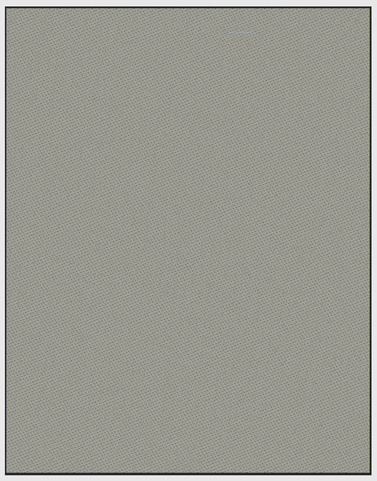
Hashes are fixed length values generated from input text. Once data is passed through a hash algorithm, the output cannot be reversed to recover the original input. If the same input is used, the same hash value will always be produced.
(Vaideeswaran, 2024)

Example
Input Text : Apple
Output Value :
f223faa96f22916294922b171a2696d868fd1f9129302eb41a45b2a2ea2ebbfd (SHA256 Algorithm)

MAC Address

A MAC address is a unique identifier assigned to a device's network **hardware**.
(GeeksforGeeks, 2024)

A created matrix



If you look closely to the top right, that highlighted text is the stored password.

Securing passwords for user accounts is a major concern in cybersecurity. Matrix-Based Password Storage (MBPS) is a method used to store passwords based on the concept of matrices. In this approach, a real password hash is stored among thousands of fake hashes in a matrix, making it difficult to identify. The coordinates of the actual hash within the matrix are generated when a new user account is created, adding an extra layer of security.

How is the coordinate generated?

Username : Southern Illinois University
Password : CARP 2026

Step 1 : Hashing and Seed Creation

Inputs
Username : Southern Illinois University
Password : CARP 2026

Hash username and password using SHA256 Algorithm.
Southern Illinois University -> 1c9ebe2b78d47106897e967123c862f367ad13bfb9c0af06fa…
CARP 2026 -> da2a0839b48915ad096891624889414ba0d8b7b7a70990b4dbe796f755c2…

Create a seed by joining the 2 hashes.
1c9ebe2b78d47106897e967123c862f367ad13bfb9c0af06fa1d4c27b576f1fd
+
da2a0839b48915ad096891624889414ba0d8b7b7a70990b4dbe796f755c2b9e4
=
1c9ebe2b78d47106897e967123c862f367ad13bfb9c0af06fa1d4c27b576f1fdda2a0839b48…

Outputs
Hashed Username
Hashed Password
Seed

Step 2 : Entity List Generation

Inputs
Seed

Obtain the system's MAC address.
MAC = D843AE60C34E

Hash the MAC address to create hash 1.
Hash 1 = bac468a9b111e1e6bb54034dde81624fd8289721bf99539c72915bd119624d19

Lowercase the MAC address.
D843AE60C34E -> d843ae60c34e

Hash the lowercase MAC address to create hash 2.
Hash 2 = 092511acddd60783186329875c222d810f250703e8879c561ce61f12e2e5096f

Join the 2 hashes together.
Hash 1 + Hash 2 = bac468a9b111e1e6bb54034dde81624fd8289721bf99539c72915bd119…

Remove all letters leaving only numbers.
Numbers = 46891111654034816248289721995397291511962419092511607831863298 7…

Shuffle the numbers using the Seed as a random seed.
Shuffled numbers = 09068918819435993221481542192526038988193522059672656 72…

Take only the first 32 numbers and segment them every 2 numbers.
32 segmented numbers : 09|06|89|18|81|94|35|99|32|21|48|15|42|19|25|26

Add all these numbers to a list called the Entity List.
Entity List : [09, 06, 89, 18, 81, 94, 35, 99, 32, 21, 48, 15, 42, 19, 25, 26]

Outputs
Entity List

Step 3 : Tape Creation

Inputs
Username Hash
Password Hash
Entity List

Establish the Entity and character list.
Entity : [09, 06, 89, 18, 81, 94, 35, 99, 32, 21, 48, 15, 42, 19, 25, 26]
Character : [a, b, c, d, e, f, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

Take the username hash and substitute the characters in the hash using the lists above. This is done through indexing. This would create the username tape.
[09, 06, 89, 18, 81, 94, 35, 99, 32, 21, 48, 15, 42, 19, 25, 26]
[a b, c, d, e, f, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

Username Hash
1c9ebe2b78d47106897e967123c862f367ad13bfb9c0af06fa1d4c27b576f1fd
V
99892581068132061925184819993542252619812642199932218925423294214219091…
That would create a 128 character long number which are called Tapes.

Repeat the same process which the password hash to create the password tape.
Password Hash : da2a0839b48915ad096891624889414ba0d8b7b7a70990b4dbe796f755…
V
Password Tape : 18093209352521260648252699150918352642252699423248252526 48…

Outputs
Username Tape
Password Tape

Step 4 : Tape Segmentation

Inputs
Username Tape
Password Tape

We segment the username and password tapes into 4 equal segments and we place them in a list. This process is called segmentation.
Username Tape : 9989268106813206192518481999354225261981264219993221892 54…
V
Username Segments
['9989268106813206192518481999354 2', '25261981264219993221892542329421', '42190918992106940626893509943542', '940999184889321906151942949994 18']

Password Tape : 18093209352521260648252699150918352642252699423248252526 48…
V
Password Segments
['18093209352521260648252699150918', '35264225269942324825252648994806', '09351825061906190919352626350648', '18068119264294191515893206268148']

We order the newly segmented username tape in the list from greatest to least to create a tuple with the segment and its original location in the tape.
Username Segments
['9989268106813206192518481999354 2', '25261981264219993221892542329421', '42190918992106940626893509943542', '940999184889321906151942949994 18']
V
[('9989268106813206192518481999354 2', 0), ('940999184889321906151942949994 18', 3), ('42190918992106940626893509943542', 2), ('25261981264219993221892542329421', 1)]

The original order of 0,1,2,4 changed to 0,3,2,1 after the segments were sorted.

Outputs
Segmented and Ordered Username Tape
Segmented Password Tape

Step 5 : Coordinate Generation

Inputs
Segmented and Ordered Username Tape
Segmented Password Tape

We take the first segment of both tapes and label them 1 and 2. We also take note of the second number from the username tape tuple. An initial coordinate is set at (1, 1)

For each segment, the system runs 32 cycles. In each cycle, it reads a value from both tapes at the same index aka cycle number. The value from Tape 2 is checked to determine if it is even or odd. If the value is even, a movement is applied to the current x and y coordinates. The type of the movement comes from Tape 1 (see graph), while the direction is determined by the segment's predefined direction that is determined by the number attached with the username tape segment. This process repeats for all 32 values in the segment.

Then after that cycle is complete you will get an initial coordinate. Once we get the initial coordinate we get its absolute value and set the coordinate as that to avoid negative coordinate values.

Then we move on to the next segment in the list. This same process is repeated for every segment meaning this will happen 4 times total. The direction will move every time. Every new cycle will add 1 to the amount of movement. This means the first cycle will have a 1 times movement multiplier. The second cycle will have a 2 times movement multiplier. The third cycle will have a 3 times movement multiplier. The last cycle will have a 4 times movement multiplier. This escalation of greater movement distances makes the coordination generation more random.

Cycle 1
Coordinate : (1, 1)
Username Segment/Tape 1 : ('9989268106813206192518481999354 2', 0)
Password Segment/Tape 2 : '18093209352521260648252699150918'
Movement Direction : Down Right
Initial Coordinate : (23, 21)
Absolute Coordinate : (23, 21)

Cycle 2
Coordinare : (23, 21)
Username Segment/Tape 1 : ('940999184889321906151942949994 18', 3)
Password Segment/Tape 2 : '35264225269942324825252648994806'
Movement Direction : Up Left
Initial Coordinate : (-23, -45)
Absolute Coordinate : (23, 45)

Cycle 3
Coordinare : (23, 45)
Username Segment/Tape 1 : ('42190918992106940626893509943542', 2)
Password Segment/Tape 2 : '09351825061906190919352626350648'
Movement Direction : Down Left
Initial Coordinate : (-13, 126)
Absolute Coordinate : (13, 126)

Cycle 4
Coordinare : (13, 126)
Username Segment/Tape 1 : ('25261981264219993221892542329421', 1)
Password Segment/Tape 2 : '18068119264294191515893206268148'
Movement Direction : Up Right
Initial Coordinate : (85, 14)
Absolute Coordinate : (85, 14)

After all the cycles we get a final coordinate of (85, 14)

Outputs
Coordinate

Step 6 : Matrix Creation and Hash Placement

Inputs
Hashed Username
Hashed Password
Coordinate

We do not place the password directly into the matrix, instead, it is first modified. This is done by combining the username and password hashes and alternating their characters to form a new string. That resulting string is then hashed again to produce a final hash, which is the hash that is stored in the matrix.

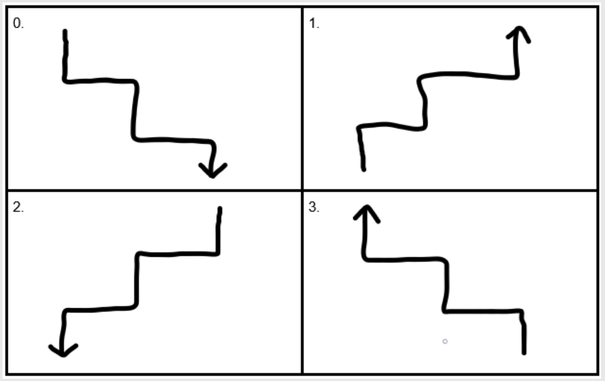
Example of character alternation
Text 1 : Banana
Text 2 : Apple
Alternated Text : BAapnpalnea

Username Hash : 1c9ebe2b78d47106897e967123c862f367ad13bfb9c0af06fa1d4c27b576…
Password Hash : da2a0839b48915ad096891624889414ba0d8b7b7a70990b4dbe796f755…
V
Alternated String : 1dca92eab0e823b97b84d84971150a6d809976e8996176122438c8896…
V
Final Hash : bb8f97162540cec84600be5c9f3b47249dc91364fe19a84eef48e6e0800ac45f

Next, a matrix is created to store the hash. A 400 × 400 matrix is generated, where each coordinate is filled with a hash derived from a fake password. Once the matrix is populated, the final hash is placed into the matrix at the coordinate that was generated earlier.

Outputs
Matrix

Movement Direction for Step 5



				9	y - 3	9			
The coordinate for placement read from Tape 1		5	4	3	y - 2	3	4	5	
		6	1	2	y - 1	2	1	6	
	8	7	0	Direction 3	y - 0	Direction 1	0	7	8
	x - 3	x - 2	x - 1	x - 0		x + 0	x + 1	x + 2	x + 3
	8	7	0	Direction 2	y + 0	Direction 0	0	7	8
		6	1	2	y + 1	2	1	6	
		5	4	3	y + 2	3	4	5	
				9	y + 3	9			

Benefits

- Usernames are never stored directly in the matrix.
- Different passwords for the same username produce different coordinates.
- Even if an attacker gains access to the matrix, they would still need to sift through 160,000 hashes.
- Highly Configurable
- Miniaturization Potential

User Authentication

User authentication is an essential component of any system that stores passwords. In Matrix-Based Password Storage (MBPS), the authentication process follows the same steps used during initial account creation. The system generates the coordinate using the provided credentials and applies the same method to produce the final hash. Instead of creating a new matrix, the system retrieves the stored hash at the generated coordinate and compares it to the hash derived from the login attempt. If the two hashes match, the user is successfully authenticated.

Citations
Password storage cheat sheet. Password Storage - OWASP Cheat Sheet Series. (n.d.). https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
Vaideeswaran, N. (2025, January 16). *What is hashing in cybersecurity?*. CrowdStrike. https://www.crowdstrike.com/en-us/cybersecurity-101/data-protection/data-hashing/
GeeksforGeeks. (2024, September 30). *What is MAC address?* https://www.geeksforgeeks.org/computer-networks/mac-address-in-computer-network/

Shealy Clifton